W1

CLASS & OBJECT: BLUEPRINT สู่ INSTANCE

Data Structures & Object-Oriented Programming



Table of Contents

- ปัญหาของ Procedural Programming
- class คือ blueprint
- Attributes vs Methods
- Anatomy of a Class
- self คืออะไร?
- _init_ คือ Constructor
- Common Mistakes

ปัญหาของ Procedural Programming

สมมติว่าเราต้องเขียนโปรแกรมจัดการบัญชีธนาคาร เขียนแบบ Procedural (แบบที่เคยเรียนมา) ได้ดังนี้

```python
# บัญชีที่ 1
owner1 = "Alice"
balance1 = 1000.0

# บัญชีที่ 2
owner2 = "Bob"
balance2 = 500.0

def deposit(balance, amount):
    if amount <= 0:
        print("จำนวนเงินต้องมากกว่า 0")
        return balance
    return balance + amount

def withdraw(balance, amount):
    if amount > balance:
        print("ยอดเงินไม่เพียงพอ")
        return balance
    return balance - amount

# การใช้งาน
balance1 = deposit(balance1, 200.0)
balance2 = withdraw(balance2, 100.0)

print(f"{owner1}: {balance1}")
print(f"{owner2}: {balance2}")
```

```
Alice: 1200.0
Bob: 400.0
```

ถ้ามีบัญชี 1,000 บัญชีจะเกิดอะไรขึ้น?

- จำนวนตัวแปรระเบิด – ต้องประกาศ owner1, balance1, owner2, balance2, ..., owner1000, balance1000 แยกกันทั้งหมด
- ข้อมูลและฟังก์ชันแยกจากกัน – ฟังก์ชัน deposit() ไม่รู้ว่าตัวเองเป็น "ของ" บัญชีไหน ต้องรับ balance ผ่าน argument ทุกครั้ง
- ไม่มีการป้องกันข้อมูล – ใครก็สามารถเขียน balance1 = -99999 โดยตรงได้โดยไม่ผ่าน validation ใดๆ
- ขยายยาก – ถ้าต้องเพิ่มข้อมูล เช่น account_number หรือ interest_rate ต้องเพิ่มตัวแปรใหม่อีกหลายร้อยตัว

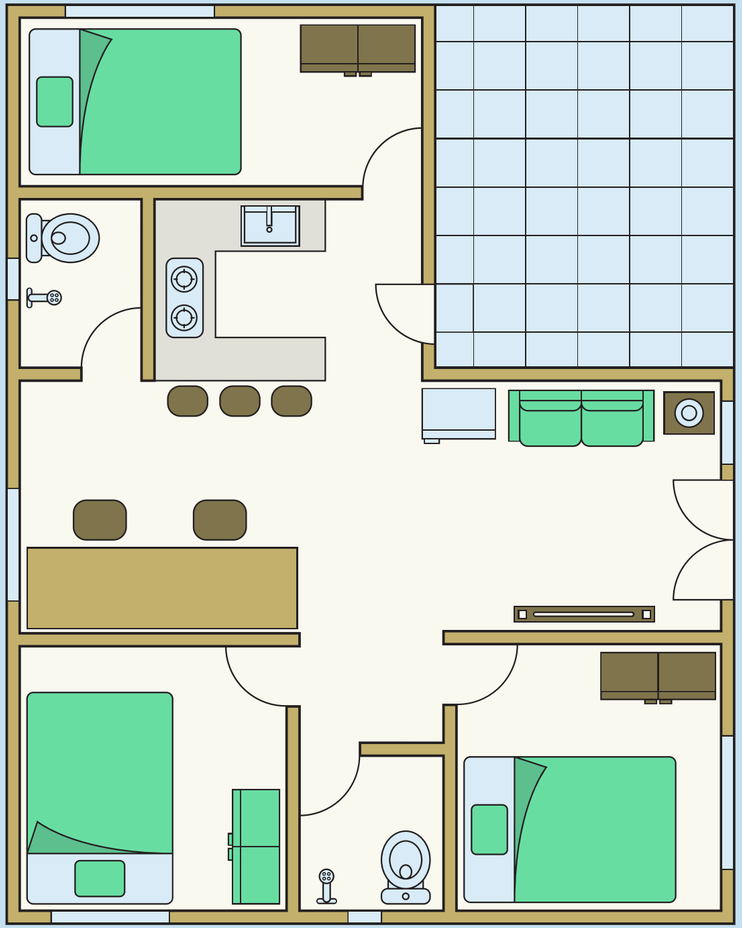
แทนที่จะจัดการข้อมูลและฟังก์ชันแยกกัน

OOP "มัดรวม" ข้อมูลและพฤติกรรมที่เกี่ยวข้องกันไว้ด้วยกันในสิ่งที่เรียกว่า Object

Class

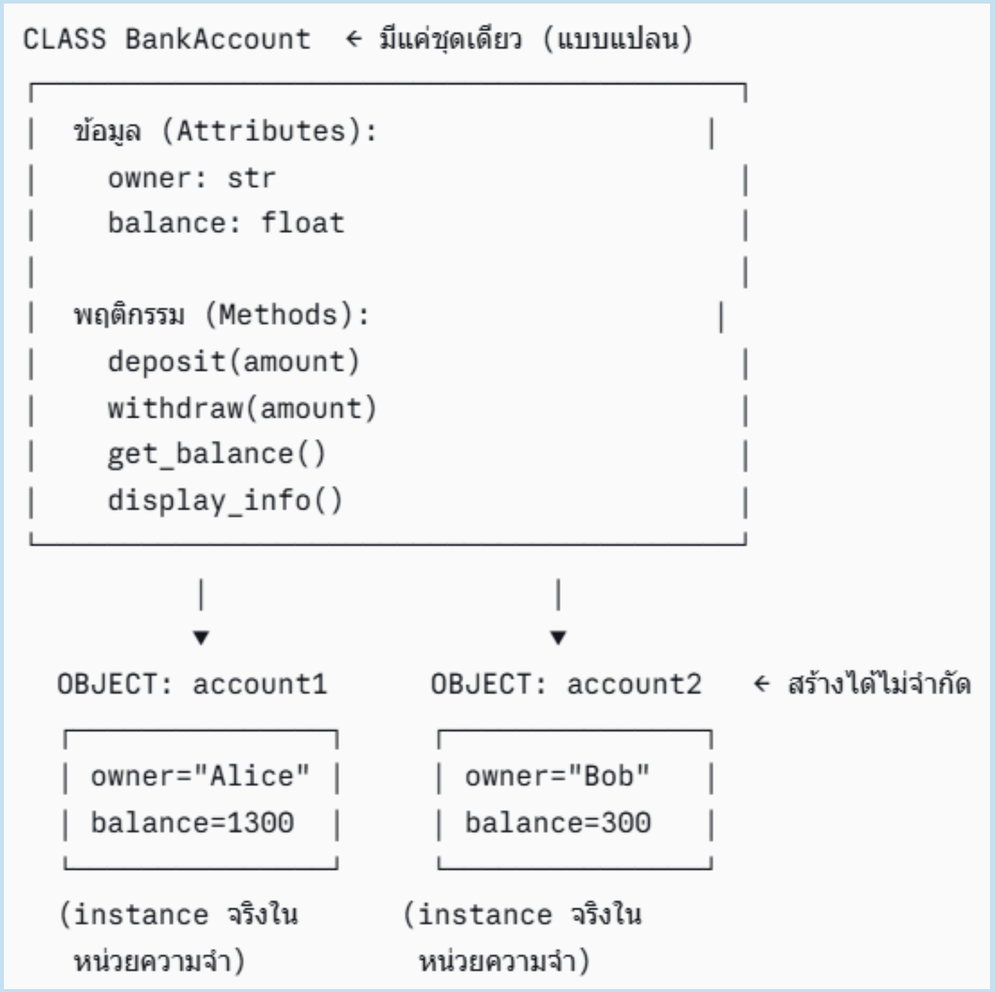
Class คือ Blueprint

- แบบแปลนบ้านไม่ใช่บ้านจริงๆ – เราอยู่ในแบบแปลนไม่ได้
  - แต่แบบแปลนบอกเราว่าบ้านที่สร้างจะมีกี่ห้อง มีหน้าต่างตรงไหน และประตูเปิดอย่างไร
- จากแบบแปลนเดียวกัน เราสามารถสร้างบ้านได้หลายหลัง และบ้านแต่ละหลังก็เป็น "บ้าน" จริงๆ ที่มีคนอยู่อาศัยได้



- ในโลก OOP: Class คือแบบแปลน (template ที่บอกว่า object ประเภทนี้มีข้อมูลอะไรและทำอะไรได้บ้าง)
- Object (Instance) คือบ้านจริงๆ (สิ่งที่สร้างขึ้นมาจาก class และมีค่าจริงๆ อยู่ในหน่วยความจำ)

Class คือ Blueprint



จุดสำคัญคือ Class มีแค่ชุดเดียว แต่ Object สร้างได้ไม่จำกัด

แต่ละ object มีค่า attributes เป็นของตัวเองโดยสมบูรณ์

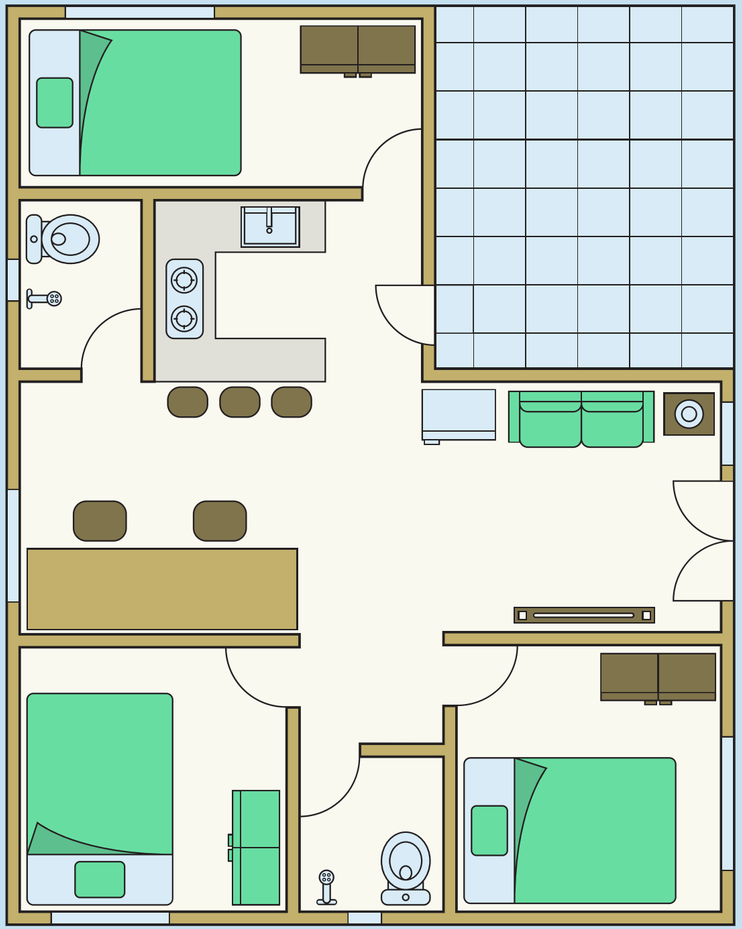
Attributes vs Methods

Attributes vs Methods

ทุก class ประกอบด้วย 2 ส่วนหลัก

1. **Attributes** คือข้อมูลที่ object นั้นเก็บไว้
    - เป็น "คุณสมบัติ" หรือ "สถานะ" ของ object
    - เช่น owner และ balance ของบัญชีธนาคาร
    - ใน Python เราเข้าถึง attribute ด้วย dot notation เช่น account1.owner หรือ account1.balance
2. **Methods** คือ สิ่งที่ object นั้นทำได้
    - เป็น "พฤติกรรม" หรือ "ความสามารถ" ของ object
    - เช่น deposit(), withdraw(), display_info()
    - Method คือฟังก์ชันที่นิยามภายใน class และมักจะอ่านหรือเปลี่ยนแปลง attributes ของ object นั้นเอง

- วิธีจำง่ายๆ:
    - Attributes = คำนาม (สิ่งที่ object มี)
    - Methods = กริยา (สิ่งที่ object ทำได้)



Anatomy of a Class

class ใน Python ประกอบด้วยองค์ประกอบหลักดังนี้

1. **class keyword และชื่อ class** ทำหน้าที่ประกาศ blueprint
    - โดยชื่อ class ในภาษา Python ใช้รูปแบบ PascalCase เสมอ เช่น BankAccount หรือ StudentRecord ไม่ใช่ bank_account หรือ bankaccount
2. **__init__ method** เป็น special method ที่ Python เรียกใช้โดยอัตโนมัติทุกครั้งที่มีการสร้าง object ใหม่
    - หน้าที่หลักของ __init__ คือการกำหนดค่าเริ่มต้นให้กับ attribute ต่างๆ ของ object นั้น
3. **self** เป็น parameter ตัวแรกของทุก method ใน class โดยไม่มีข้อยกเว้น
    - self ทำหน้าที่อ้างอิงถึง object ที่กำลังถูกใช้งานในขณะนั้น
    - เปรียบได้กับสรรพนาม "ตัวเอง" ที่ทำให้แต่ละ object รู้ว่ากำลังพูดถึงข้อมูลของตนเอง ไม่ใช่ของ object อื่น
4. **instance attributes** คือ ข้อมูลที่แต่ละ object เก็บเป็นของตัวเอง
    - กำหนดผ่าน self.ชื่อ_attribute ภายใน __init__ และสามารถเข้าถึงได้จากทุก method ภายใน class เดียวกัน
5. **methods** คือ ฟังก์ชันที่อยู่ภายใน class และสามารถเข้าถึง attribute ของ object ได้โดยตรงผ่าน self
    - methods เหล่านี้คือพฤติกรรมที่ object สามารถทำได้

Ex: Anatomy of a Class

```python
class BankAccount:

    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def get_balance(self):
        return self.balance
```

class keyword + ชื่อ class (PascalCase)

__init__: กำหนดค่าเริ่มต้น
instance attribute
instance attribute

method สำหรับฝากเงินเข้าบัญชี

method คืนยอดเงินคงเหลือในบัญชี

self คืออะไร?

- เมื่อเราสร้าง account1 และ account2 จาก class BankAccount เดียวกัน และเรียก account1.deposit(200)
  - Python จะต้องรู้ว่า "deposit 200 เข้าบัญชีไหน?"
- คำตอบคือ self ซึ่งเป็น การอ้างอิงถึง object ที่ถูกเรียก method ในขณะนั้น

- เมื่อเราเขียน account1.deposit(200) Python แปลสิ่งนี้เบื้องหลังเป็น:

  BankAccount.deposit(account1, 200)

```
account1.deposit(200)
    |
    |  Python ส่ง account1 เป็น self โดยอัตโนมัติ
    ▼
def deposit(self, amount):          ← self = account1
    self.balance += amount          ← account1.balance += 200
```

Python ส่ง account1 เข้ามาเป็น argument แรกโดยอัตโนมัติ
และ parameter ตัวแรกของ method (ซึ่งเรานิยามว่า self โดยธรรมเนียม) จะรับค่านั้นไว้

ดังนั้นเมื่ออยู่ใน method และเขียน self.balance += amount
หมายความว่า "เพิ่ม amount เข้าใน balance ของ object ตัวนี้

กฎสำคัญ: self ต้องเป็น parameter ตัวแรกของทุก method ใน class
ถ้าลืม จะเกิด TypeError ทันทีเมื่อเรียก method นั้น

__init__ คือ Constructor

- _init_ คือ method พิเศษที่ Python เรียกโดยอัตโนมัติทุกครั้งที่เราสร้าง object ใหม่ จาก class นั้น

- หน้าที่ของ _init_ คือ ตั้งค่าเริ่มต้น (initialize) ให้กับ attributes ของ object ที่เพิ่งถูกสร้างขึ้น
  - เหมือนกับช่างที่เข้าไปตกแต่งบ้านใหม่ทันทีหลังจากสร้างเสร็จ ก่อนที่เจ้าของจะย้ายเข้ามา

- เมื่อเราเขียน account1 = BankAccount("Alice", 1000.0) Python ทำตามขั้นตอนเหล่านี้:
  - สร้าง object ว่างๆ ใหม่ในหน่วยความจำ
  - เรียก _init_ โดยส่ง object นั้นเป็น self และส่ง "Alice" กับ 1000.0 เป็น arguments
  - _init_ สร้าง self.owner = "Alice" และ self.balance = 1000.0 บน object นั้น
  - คืน object ที่พร้อมใช้งานแล้วให้กับตัวแปร account1

ข้อสังเกตสำคัญ: _init_ ไม่ได้สร้าง object

Python สร้าง object ก่อนแล้วค่อยเรียก _init_ เพื่อ "ตั้งค่า" object นั้น

Instantiation

- กระบวนการสร้าง object (instance) จาก class
- โดย syntax ใน Python ตรงไปตรงมา

รูปแบบ: ตัวแปร = ClassName(arguments)

```
account1 = BankAccount("Alice", 1000.0)
account2 = BankAccount("Bob", 500.0)
account3 = BankAccount("Charlie")
```

balance เริ่มต้นที่ 0.0 (default value)

หลังจาก instantiation:

- account1, account2, account3 เป็น object สามตัวที่แยกกันในหน่วยความจำ
- การเปลี่ยน account1.balance ไม่ส่งผลต่อ account2.balance เลย
- ทุก object ใช้ methods จาก class เดียวกัน แต่ทำงานกับข้อมูลของตัวเอง

ฝึกเขียนโปรแกรม



Live Coding: สร้างระบบบัญชีธนาคารอย่างง่าย

- โจทย์: เราจะสร้าง class BankAccount เพื่อจำลองบัญชีธนาคาร 1 บัญชี

- ข้อมูลที่บัญชีต้องจำ (Attributes)
  - ชื่อเจ้าของบัญชี (owner)
  - ยอดเงินคงเหลือ (balance) – เริ่มต้นที่ 0 ถ้าไม่ระบุ

- สิ่งที่บัญชีต้องทำได้ (Methods)
  - deposit(amount) – ฝากเงิน (amount ต้องมากกว่า 0)
  - withdraw(amount) – ถอนเงิน (ห้ามถอนเกินยอดที่มี)
  - get_balance() – คืนค่ายอดเงินปัจจุบัน
  - display_info() – แสดงข้อมูลบัญชีในรูปแบบที่อ่านง่าย

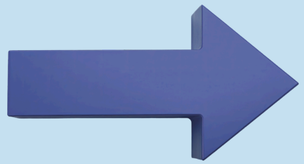
Live Coding: สร้างระบบบัญชีธนาคารอย่างง่าย (2)

```python
# --- ทดสอบการทำงาน ---
account1 = BankAccount("Alice", 1000.0)
account2 = BankAccount("Bob")

account1.display_info()
account1.deposit(500.0)
account1.withdraw(200.0)
account1.withdraw(5000.0)

account2.deposit(300.0)
account2.display_info()

print(f"\nยอดของ Alice: {account1.get_balance():.2f}")
print(f"ยอดของ Bob:   {account2.get_balance():.2f}")
```



ผลลัพธ์ที่คาดหวัง

```
เจ้าของ: Alice | ยอดเงิน: 1000.00 บาท
ฝากเงิน 500.00 บาท | ยอดคงเหลือ: 1500.00 บาท
ถอนเงิน 200.00 บาท | ยอดคงเหลือ: 1300.00 บาท
ยอดเงินไม่เพียงพอ
ฝากเงิน 300.00 บาท | ยอดคงเหลือ: 300.00 บาท
เจ้าของ: Bob | ยอดเงิน: 300.00 บาท


ยอดของ Alice: 1300.00
ยอดของ Bob:   300.00
```

โจทย์เสริม: ต่อยอดจาก Alice และ Bob

1) สร้าง object เพิ่ม
โดยสร้างบัญชีที่สามชื่อ Charlie ด้วยยอดเริ่มต้น 2000 บาท แล้วแสดงข้อมูลด้วย display_info()

```
เจ้าของ: Charlie | ยอดเงิน: 2000.00 บาท
```

2) โอนเงินระหว่าง object
เขียน code ที่ "โอนเงิน" 500 บาท จากบัญชี Alice ไปยังบัญชี Bob โดยใช้ method ที่มีอยู่แล้ว (ไม่ต้องเพิ่ม method ใหม่) แล้วแสดงยอดของทั้งคู่

```
ยอดของ Alice: 800.00
ยอดของ Bob: 800.00
```

3) วนซ้ำกับหลาย object
เก็บบัญชีทั้งสาม (Alice, Bob, Charlie) ไว้ใน list แล้วใช้ for loop แสดงข้อมูลของทุกบัญชีในครั้งเดียว

```
#เจ้าของ: Alice | ยอดเงิน: 800.00 บาท
#เจ้าของ: Bob | ยอดเงิน: 800.00 บาท
#เจ้าของ: Charlie | ยอดเงิน: 2000.00 บาท
```

Instance Independence

- เมื่อสร้าง object จาก class Python จะจัดสรรพื้นที่หน่วยความจำใหม่ให้กับ object แต่ละตัวอย่างแยกอิสระจากกัน
- โดย class ทำหน้าที่เพียง blueprint หรือแม่พิมพ์ที่บอกว่า object ที่สร้างขึ้นมาจะมีโครงสร้างอย่างไร แต่ข้อมูลที่เก็บอยู่ภายในแต่ละ object นั้นเป็นของ object นั้นโดยเฉพาะ

```
alice = BankAccount("Alice", 1000)
bob   = BankAccount("Bob",    500)

alice.deposit(200)

print(alice.get_balance())  # 1200 — เฉพาะของ Alice ที่เปลี่ยน
print(bob.get_balance())    # 500  — Bob ไม่ได้รับผลกระทบใดๆ
```

การฝากเงินเข้าบัญชีของ Alice ไม่มีผลต่อบัญชีของ Bob แม้ว่าทั้งคู่จะถูกสร้างจาก BankAccount class เดียวกันก็ตาม เพราะ self.balance ของ Alice และ self.balance ของ Bob คือพื้นที่หน่วยความจำคนละตำแหน่งกันโดยสิ้นเชิง

Object Lifecycle

- object ใน Python ไม่ได้ "อยู่" ในโปรแกรมตลอดเวลา แต่มีวงจรชีวิตที่ชัดเจน
- วงจรชีวิตของ object แบ่งออกเป็น 3 ช่วงหลัก ดังนี้
  - **ช่วงที่ 1 — การเกิด (Instantiation)**
    - เกิดขึ้นเมื่อเรียกใช้ class เหมือนการเรียกฟังก์ชัน เช่น alice = BankAccount("Alice", 1000)
    - Python จะจัดสรรหน่วยความจำให้กับ object ใหม่ จากนั้นเรียก __init__ โดยอัตโนมัติเพื่อกำหนดค่าเริ่มต้น และ object นั้นจะถูกผูกกับตัวแปร alice
  - **ช่วงที่ 2 — การใช้งาน (Active Use)**
    - ช่วงที่โปรแกรมเรียกใช้ method ต่างๆ ของ object เพื่ออ่านหรือเปลี่ยนแปลงข้อมูล
    - ตราบใดที่ตัวแปรยังอ้างอิงถึง object อยู่ Python จะเก็บ object นั้นไว้ในหน่วยความจำ
  - **ช่วงที่ 3 — การสิ้นสุด (Garbage Collection)**
    - เกิดขึ้นเมื่อไม่มีตัวแปรใดอ้างอิงถึง object นั้นอีกต่อไป ไม่ว่าจะเกิดจากการกำหนดค่าใหม่ให้ตัวแปร การออกจาก scope ของฟังก์ชัน หรือการกำหนดค่า None
    - Python มีระบบ Garbage Collector ที่คืนหน่วยความจำจาก object เหล่านั้นโดยอัตโนมัติ

Coding Convention (Naming & Style)

- code ที่ดีต้องสื่อสารเจตนาของผู้เขียนให้คนอื่น รวมถึงตัวเองในอนาคต สามารถอ่านและเข้าใจได้โดยง่าย
- Python มีข้อตกลงการเขียนโค้ดที่เป็นมาตรฐานที่นิยมใช้กันทั่วโลก เรียกว่า PEP 8 ซึ่งนิสิตควรทำความคุ้นเคยตั้งแต่เริ่มต้น
- ข้อตกลงที่สำคัญสำหรับการเขียน class มีดังนี้
  a. **การตั้งชื่อ**
     - ชื่อ class ใช้ PascalCase เช่น BankAccount, StudentRecord
     - ชื่อ method และ attribute ใช้ snake_case เช่น get_balance, account_number
     - ชื่อที่ดีควรสื่อความหมายได้ในตัวเอง โดยไม่จำเป็นต้องอ่าน comment ประกอบ
  b. **การเว้นบรรทัด**
     - ระหว่าง method แต่ละตัวภายใน class ควรเว้น 1 บรรทัด
     - ระหว่าง class กับโค้ดส่วนอื่นควรเว้น 2 บรรทัด เพื่อให้โครงสร้างโปรแกรมอ่านได้ชัดเจนขึ้น
  c. **Type Hints**
     - เป็นการระบุชนิดข้อมูลของ parameter และค่าที่ method คืนกลับมา
     - แม้ Python จะไม่บังคับ แต่การใส่ type hint ช่วยให้ผู้อ่านโค้ดเข้าใจการใช้งานได้ทันที และช่วยให้ IDE สามารถแจ้งเตือนข้อผิดพลาดได้ก่อนรันโปรแกรม

Ex: Coding Convention (Naming & Style)

ไม่แนะนำ – อ่านเจตนาได้ยาก

```
class bankaccount:
    def __init__(self,o,b):
        self.o=o
        self.b=b
    def dep(self,a):
        self.b+=a
```

แนะนำ – อ่านเจตนาได้ชัดเจน

```
class BankAccount:
    def __init__(self, owner: str, balance: float = 0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount: float) -> None:
        self.balance += amount
```

โค้ดทั้งสองตัวอย่างทำงานได้ผลลัพธ์เหมือนกันทุกประการ แต่ตัวอย่างที่สองสื่อสารเจตนาของผู้เขียนได้ชัดเจนกว่าอย่างเห็นได้ชัด

การฝึกนิสัยการเขียนโค้ดที่ดีตั้งแต่ต้นจะเป็นประโยชน์อย่างยิ่งเมื่อโปรแกรมมีขนาดใหญ่ขึ้นในสัปดาห์ถัดๆ ไป

Class vs Dictionary

Class vs Dictionary

ทำไมไม่ใช้ dict แทน?

แนวทาง Dictionary

```
account_dict = {
    "owner": "Alice",
    "balance": 1000.0
}

# ไม่มีอะไรป้องกันการแก้ข้อมูลโดยตรง
account_dict["balance"] = -99999.0    # ยอดติดลบ — ไม่มี error เลย
account_dict["typo_kye"] = "oops"     # เพิ่ม key ผิดๆ — ไม่มี error เลย
del account_dict["owner"]             # ลบ key สำคัญ — ไม่มี error เลย

# ถ้ามี 100 บัญชี ต้องเขียน validation logic ซ้ำทุกที่
account_dict2 = {"owner": "Bob", "balance": 500.0}
account_dict2["balance"] += 200.0     # copy logic อีกครั้ง
```

แนวทาง Class

```
account_obj = BankAccount("Alice", 1000.0)

# method มี validation ในตัว ใช้ซ้ำได้ทันที
account_obj.deposit(500.0)      # ✓ OK
account_obj.deposit(-100.0)     # ✓ จัดการ error เองอัตโนมัติ
account_obj.withdraw(99999.0)   # ✓ จัดการ error เองอัตโนมัติ

# สร้าง object ใหม่ได้ทันที — logic ไม่ต้อง copy
account_obj2 = BankAccount("Bob", 500.0)
account_obj2.deposit(200.0)     # ใช้ method เดียวกัน
```

ถ้าข้อมูลนั้น "ทำอะไรได้" → ใช้ Class ถ้าข้อมูลนั้น "แค่เก็บไว้" → ใช้ Dictionary ก็พอ

นักพัฒนาที่ดีรู้จัก เลือก tool ให้ตรงงาน

สรุป : Class vs Dictionary

	Dictionary	Class
โครงสร้างข้อมูล	มีความยืดหยุ่นสูง สามารถเพิ่มหรือลด key ได้อย่างอิสระในขณะ runtime	โครงสร้างถูกกำหนดไว้อย่างชัดเจนตั้งแต่ขั้นตอนการออกแบบ ทุก instance มีรูปแบบเดียวกัน
ความปลอดภัยของข้อมูล	ไม่มีกลไกตรวจสอบความถูกต้อง การเข้าถึง key ที่ไม่มีอยู่จะทราบผลเมื่อโปรแกรมทำงานเท่านั้น	สามารถกำหนด validation logic ไว้ภายใน method เพื่อตรวจสอบและควบคุมข้อมูลก่อนรับหรือแก้ไขค่า
พฤติกรรมของข้อมูล	ทำหน้าที่จัดเก็บข้อมูลเพียงอย่างเดียว ไม่มีความสามารถในการประมวลผลในตัว	รวมข้อมูลและพฤติกรรม (method) ไว้ในหน่วยเดียวกัน สามารถกำหนดการกระทำที่เกี่ยวข้องได้โดยตรง
ความเหมาะสมในการใช้งาน	เหมาะสำหรับข้อมูลชั่วคราว ค่า configuration หรือข้อมูลที่รับมาจากภายนอก เช่น JSON response	เหมาะสำหรับ entity ที่มีวงจรชีวิต มีพฤติกรรมเฉพาะ และต้องการนำกลับมาใช้ซ้ำในหลายส่วนของโปรแกรม

Common Mistakes

ผิดพลาดที่ 1: ลืม self ใน method signature

ผิด

```
class BankAccount:
    def deposit(amount):
        self.balance += amount
```

← ขาด self

← self ไม่ถูกนิยาม

ถูก

```
class BankAccount:
    def deposit(self, amount):
        self.balance += amount
```

- เมื่อเรียก account.deposit(200) Python จะส่ง account เป็น argument แรกโดยอัตโนมัติ
- ถ้า method นิยามแค่ deposit(amount) (รับแค่ 1 parameter) แล้วถูกเรียกด้วย 2 ค่า
  - Python จะแจ้ง TypeError: BankAccount.deposit() takes 1 positional argument but 2 were given

ผิดพลาดที่ 2: ลืม self. หน้า attribute ใน __init__

ผิด : สร้างตัวแปร local ชั่วคราว ไม่ใช่ attribute ของ object

```
class BankAccount:
    def __init__(self, owner, balance):
        owner = owner
        balance = balance
```

← ตัวแปร local ที่หายไปเมื่อ _init_ จบ

← ไม่ได้ผูกกับ object เลย

ถูก : ผูกเข้ากับ object ด้วย self.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
```

ผิดพลาดที่ 3: เรียก method บน Class ตรงๆ

ผิด

```
BankAccount.deposit(200)
```

← TypeError: ไม่มี self

ถูก : ต้อง instantiate ก่อนเสมอ

```
account = BankAccount("Alice", 1000.0)
account.deposit(200)
```

ผิดพลาดที่ 4: ใส่ return ที่มีค่าใน __init__

ผิด

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
        return self
```

← TypeError: _init_() should return None

ถูก : ไม่ต้องมี return statement เลย

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
```

Python จัดการส่งคืน object ให้เองโดยอัตโนมัติหลังจาก _init_ ทำงานเสร็จ

ผิดพลาดที่ 5: ลืมว่า object แต่ละตัวเป็นอิสระจากกัน

```
account1 = BankAccount("Alice", 1000.0)
account2 = account1

account2.deposit(500.0)
print(account1.get_balance())
```

← คิดว่าสร้าง "สำเนา" ใหม่

1500.0 ← ไม่ใช่ 1000.0 !

- การเขียน account2 = account1 ไม่ได้สร้าง object ใหม่
  - account2 และ account1 ชี้ไปที่ object เดียวกัน ในหน่วยความจำ
- ถ้าต้องการ object ใหม่ที่มีค่าเดียวกัน ต้อง instantiate ใหม่:
  - account2 = BankAccount("Alice", 1000.0)